Project CoFI

Krishith V

June 2026

1 Microcontrollers

1.1 Subsystems

The main subsystems on a microcontroller are the CPU, clocks, SRAM and NVM. The MSPM0L2228 however has more features such as an ADC and hardware accelerators for various functions. We will focus on the main subsystems tho:

1.1.1 CPU

The Central Processing Unit is where code is actually *executed*. The MSPM0L2228 uses an Arm Cortex-M0+ CPU. This is a energy efficient CPU that runs on a clock cycle of 32MHz

1.1.2 Clocks

Clocks are the reason the entire microcontroller remains in sync with other subsystems. These provide a pulse at a rate of say 32MHz (because the CPU uses this), so other subsystems know when to give information to the CPU so that the data is not sent at a time when the CPU is not active. The MSPM0L2228 has two internal oscillators, one running at 4MHz to 32Mhz and another one at 32kHz.

1.1.3 SRAM

SRAM is where the CPU stores its current working data or the data that it is processing right now such as the call stack or the local variables. The MSPM0L2228 has 32kB of ECC protected SRAM so in case the SRAM is bit flipped or "glitched", the ECC kicks in and fixes the flipped bit.

1.1.4 NVM

Non volatile memory is where the CPU stores its long term data or data that needs to be accessed later on even after a power cycle. Because SRAM loses its data when the microcontroller is powered off, important data is stored in the

NVM for future access. The MSPM0L2228 has 256kB of flash memory with ECC.

1.2 Computation

1.2.1 Internals of a CPU

The CPU is made of a few major block such as the Register File, ALU, Control Unit and Memory interface.

Register File This is a small set of very fast storage locations that are inside the CPU. In the Cortex M0+, there are 16 standard registers and 3 special registers

ALU This is the Arithmetic Logic Unit and this is where the operations are performed (add, subtract, AND, OR, XOR, etc).

Control Unit This basically tells the ALU *what* to do. It decodes instructions that it receives and tells the ALU what operation to perform, what registers it should read/write tell the memory interface when to load or store

Memory Interface This is the connection to the SRAM and flash. The CU sends signals to this interface to load/store data

1.2.2 Register

As mentioned before there are 16 standard registers and 3 special registers. There are 13 General purpose registers which are split into two groups - Low registers (`R0-R7`) and High registers (`R8-R12`). The stack pointer is in `R13` and the link register is `R14`. The next instruction to be executed is in the Program Counter `R15`. Then there are three special registers: Program Status Register (PSR), Interrupt Mask Register (PRIMASK), and Control Register (CONTROL).
The Program Status Register is a combination of Application Status, Interrput Status, and Execution status. The application status contains the N, Z, C, and V flags to evaluate conditional branch statements. The interrupt status tells whether the CPU is running normal code or an exception handler. The Execution status tells whether the CPU is in Thumb state
The PRIMASK register is used to tell the CPU to enable/disable interrupts to make sure the current instruction takes top most priority over other instructions. The CONTROL register is used to define whether the code that is being executed is privileged or unprivileged.

1.2.3 Opcodes

The Opcode is how the Control Unit knows which control signals to send for the CPU to perform. For example if `0001` is assigned to the `ADD` function, then whenever the Control Unit gets this opcode, it knows where the operands are, and sends a sequence of instructions such as telling the ALU what to do, enabling register and memory read/write and setting the necessary flags.

1.2.4 How a CPU does math

The CPU does arithmetic operations using an ALU. The control unit sends signals to this ALU to perform these operations. The register file also gets signals from the control unit telling where to read the register values from and places these values into the input for the ALU. The ALU gives the value back to the register file to write into the output register, and it updates the necessary flags as well. To move data, the CPU uses the `LDR` and `STR` instructions which load and store data respectively. The `LDR` instruction reads data from the SRAM and places it into a register, and `STR` does the exact opposite.

1.2.5 What the CPU does for ADD

The control unit first sees the opcode and identifies the operation is ADD. Now the control unit routes the content from the registers `R2` and `R3` to the ALU. Then the output from the ALU is written into the register `R1`.

1.2.6 Fetch-Decode-Execute

This is the "cycle" that a CPU keeps performing for an instruction. It first "fetches" an instruction from the memory and write it to the Instruction register. Now the control unit "decodes" the instruction from the PC and sends the control signals to the corresponding subsystems. These subsystems, based on the signals from the Control Unit, "execute" the command.

1.2.7 Pipelining

This is an optimisation method where we overlap the Fetch-Decode-Execute instructions with other instructions. In a simple CPU, the "execute" part of an instruction finishes and then only the "fetch" part of the next instruction starts, but this wastes a lot of time and resources as the CPU is not fully utilised. To utilise the CPU fully, we perform the "fetch" part of the next instruction while say the "decode" part of the previous instruction is still executing.

1.3 Firmware

1.3.1 Flashing

Flashing is the process of sending a compiled code to the microcontroller's flash so that it can execute the code. This is usually done by connecting the computer

to a hardware debugger which is connected to the MCU's J3 port. Now it interfaces over ARM's Serial Wire Debug to send data to the MCU. The chip can be reflashed however many times you want, unless there's a flag enabled which prevents the MCU from being reflashed ever again. If we want to avoid TI's tools, we first need to find a hardware debugger that interfaces over SWD and use an open source tool such as OpenOCD or other tools. Also going through the docs, the MSPM0L2228 has somehting called a Bootstrap Loader (BSL) which directly allows us to flash the MCU over UART or I2C.

1.3.2 Spacious Addressing

This extra address space is used for a feature called Memory Mapped Input/Output (MMIO). The ARM Cortex M0 does not have special instructions for different peripherals, it simply calls a Load or a Store instruction to an address. Now the system's Bus Matrix sees this instruction, intercepts it, figures out what writing to that address means, and triggers the physical pin.

1.3.3 Time is meaningless

There are multiple ways to make an LED blink exactly at $1Hz$. One would be software wise, so basically tell the CPU to waste a specific number of cycles such that the correct time has passed. For example, here we need the CPU to be busy for $500ms$ and since the clock frequency for the CPU is $32MHz$, we need to waste $32 \times 10^6 \times 500 \times 10^{-3} = 16 \times 10^6$ cycles. This is however highly inefficient because the CPU cant do anything else during this time.
We could also use the hardware timer directly, since the clock cycle is $32MHz$, we can use a clock divider to get exactly $1Hz$. So now whenever it hits the timer, it interrupts the currently executing instruction, toggle the LED state, and go back to executing the next instruction while resetting the timer.

2 PCB

2.1 Designing the schematics

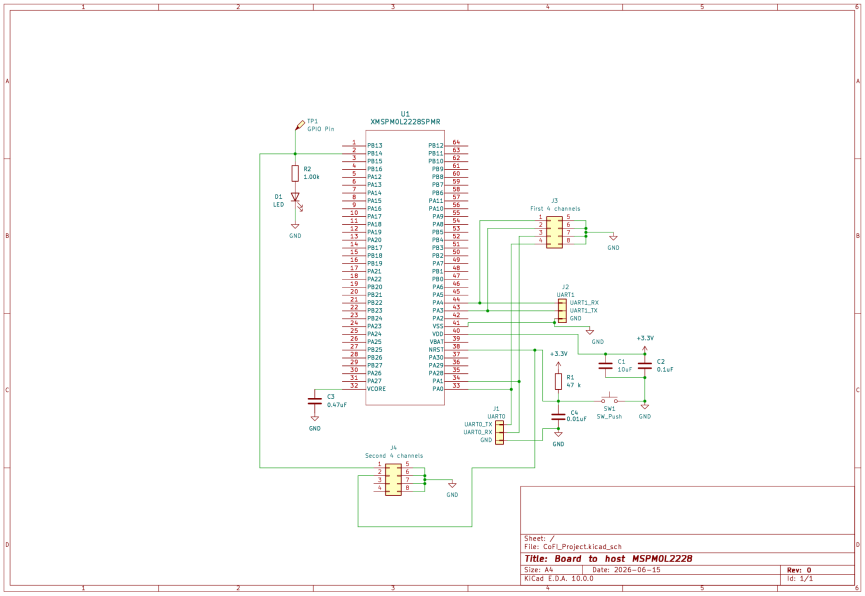

Figure 1: Schematic

Some important things here are I have used pins `PA0` and `PA1` for `UART0_TX` and `UART0_RX` respectively. Now I chose to exclude the `RTS` and `CTS` pins for the UART header. For the Saelae logic analyser I have given two 4 channel headers as that is how the real thing works, and I have connected the remaining side to `GND`. The high-res pdf is here.

2.2 Printing the board

We can use online services such as JLCPCB or PCBWay to get the PCB printed out. We send a gerber file which they will print out and ship to us. These are many features we can customise such as PCB thickness, PCB color, silkscreen, PCB material, and surface finish. PCB thickness is usually made thinner for lightweight applications such as drones or smth, PCB color and silkscreens are just vanity features and not worth the money. However surface finish is important because we can choose a type called ENIG (Elctroless Nickel Immersion Gold). This gives a flat surface so that we can solder much more easily compared to the traditional HASL.

3 Hardware Security

3.1 Fault Injection

Fault injection is creating anomalies so that the MCU performs a forbidden operation. This can be done using various methods such as:[1]

Voltage glitching Here we momentarily either increase or decrease V_{dd} so this changes the speed of the transistor switching because the speed at which it switches is directly proprotional to the supply voltage. Say if you momentarily drop the V_{dd}, the setup and hold times of flip-flops increases and this makes it capture the data at a wrong time. However this "glitch" should be short enough to prevent a Brown Out Reset. This is when if the V_{dd} is low enough for a long enough time, it resets the chip. This voltage glitch usually is around the $10ns$ to $1\mu s$ range. Similarly, you can do the same thing with increasing V_{dd}

Clock glitching Here we corrupt the clock signal. By modifying the clock signal, we could use the fact that the data signal might have changed in that extra time period, so we could feed the flip flop some other data and cause a setup time violation.

Electromagnetic Fault Injection Here a small electromagnetic pulse is produced which directly disturbs the power distribution inside the chip. This is "technically" the same principle as Voltage glitching, but usually capacitors on the board smooth out the input that you give. With EMFI, we are directly introducing the disturbance on the specific region. This "specificness" allows us to affect only specific blocks instead of the entire chip. However the drawback with this method, is that you need to do careful positioning as you don't want to disturb some other regions.

Laser Fault Injection Here a focused laser is directed at a specific transistor or a memory cell. Here the photons create electron hole pairs in the chip, which can momentarily forward bias a junction or flip the state of a SRAM cell.

[1] Do note, this section was taken from my Coordinator application

3.2 Voltage Glitching

Based on the data sheet for the microprocessor I guess the points of glitching would be in V_{dd}. However on the datasheet of the board given, I would assume the point of glitching to be the `3v3` pins in one of the `J1`, `J2`, or `J3` headers. I believe the best pin would be the one in `J3` tho because it is all connected to the debugger here so we can connect a ChipWhisperer here.

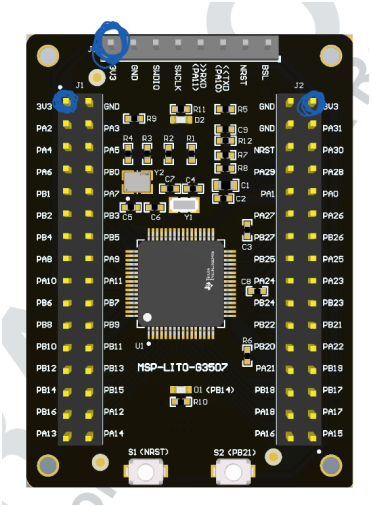

Figure 2: Glitch points

However we could also use lift the Vdd pin on the microprocessor and glitch there

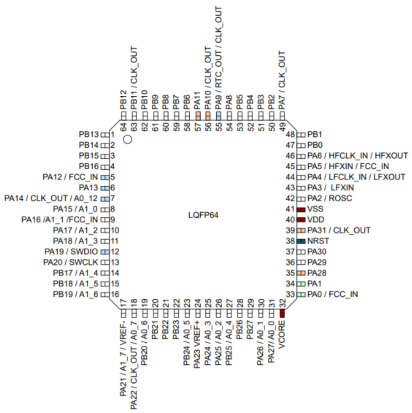

Figure 3: Pin 40 here is Vdd

To actually perform the crowbar glitch we will add a MOSFET, connect the drain to V_{dd} and the source to a capacitor connected to ground (This capacitor is placed as close as possible to the MOSFET so there is as little time delay as possible) and an FPGA will generate the pulse for the MOSFET.

3.3 Exploiting Voltage Glitches

This is the `check_function` compiled using ARM clang compiler

```
bl      _Z9check_pinPKc
cmp     r0, #0
beq     .LBB1_4
```

Here we can see that if we glitch the `beq` instruction, we can easily gain accesse even with the incorrect pin. We could also glitch here:

```
bl      strcmp
cmp     r0, #0
ldr     r0, .LCPI0_0
moveq   r0, #1
```

Here if we glitch the `cmp` instruction, we could confuse the `moveq` instruction (because it reads the Z flag, so if the `cmp` instruction is corrupted, the flag might not be set correctly) so it will assign 1 to the output (basically telling that the pin was correct) so it will grant access.
The main issue with glitching is the fact that its not exactly deterministic as even the tiniest timing change will ruin the glitch, so we need feedback on what the code is doing while glitching. To get this feedback we could do some power analysis to see which branch was taken, or we could attach a GPIO pin to these blocks, so when a branch is chosen it powers that pin.

4 Chip Whisperer

4.1 Setup for the CW Nano

Initially we need to set up the platform and stuff and this is done by `cw.scope()`. This will autodetect the device that you're using. Now we have to actually conenct to the device and this is done by `cw.target(scope)`. The default target type here is `SimpleSerial` and that is why we are choosing to omit it in the `target()` function. Now we will finally call `scope.default_setup()` and this does the following thing:

1. Sets the scope gain to 45dB
2. Sets the scope to capture 5000 samples
3. Sets the scope offset to 0 (aka it will begin capturing as soon as it is triggered)
4. Sets the scope trigger to rising edge
5. Outputs a 7.37MHz clock to the target on HS2
6. Clocks the scope ADC at 4*7.37MHz. Note that this is synchronous to the target clock on HS2
7. Assigns GPIO1 as serial RX
8. Assigns GPIO2 as serial TX

However this is set for the CWLite/CW1200, but since we are using the CW Nano, we have to set the the clock frequency to 7.5MHz. Now to upload the firmware we have to compile the firmware using the build system already provided in the ChipWhisperer, and we can upload the firmware using the function `.program_target()`
Now to communicate with the target we could either use raw serial or use SimpleSerial commands. We will use the simpleserial commands for an example. To write we can call the function `target.simpleserial_write('p', msg)` which will send the bytearray of `msg` as an input. Similarly we can read the output using `target.simpleserial_read('r', 16)`. The final pseudocode will be smth like this:

```
scope = cw.scope()
scope.default_setup()

scope.io.clkout = 7.5E6
target = cw.target(scope)

#To upload the firmware
fw_path = "path.hex"
cw.program_target(scope, cw.programmers.STM32FProgrammer, fw_path)
```

```
# The STM32FProgrammer is used because the CW Nano uses an STM32

#Reboot the target after flashing
def reboot_flush():
    scope.io.nrst = False
    time.sleep(0.05)
    scope.io.nrst = "high_z"
    time.sleep(0.05)
    #Flush garbage too
    target.flush()

reboot_flush()

# To send and receive data
#Send input
msg = bytearray([0] * 16)
target.flush()
target.simpleserial_write('p', msg)

# Read output
response = target.simpleserial_read('r', 16)
print(response)
```

4.2 Voltage Glitching on a CW Husky

First we set the husky up:

```
scope.clock.clkgen_src = 'system'
scope.clock.clkgen_freq = 10e6
scope.clock.adc_mul = 1

scope.adc.basic_mode = "rising_edge"

scope.trigger.triggers = "tio4"
scope.io.hs2 = "clkgen"
```

The glitch generation is disabled by default, so we have to turn it on:

```
scope.glitch.enabled = True
scope.glitch.clk_src = 'pll'
scope.clock.fpga_vco_freq = 600e6
scope.glitch.output = 'glitch_only'
scope.glitch.trigger_src = 'manual'
scope.glitch.repeat = 1
```

Now we can actually perform our glitch. We will be setting the offset values and the width value and trigger by calling `manual_trigger()`. Now to get a succesful glitch we sweep through these values and glitch it to see if we get the

behaviour we need. The glitch here is performed after a specific offset. However we can also set the trigger source to be an external pin, so whenever it flips high, it starts the offset and fires the glitch.

5 Side Channel Attacks

These are a really cool passive way of exploiting a system as you're simply decoding information that you are already getting. Say for example a chip is doing some calculations, we can figure out what calculation it is doing simply by seeing how much power its consuming or checking how long it takes for it to run the program. Some examples of side channel attacks are power analysis attacks, timing attacks, electromagnetic attacks and acoustic attacks (these are all pretty self explanatory, we are basically. I will discuss a bit more about power analysis and how Hamming weight is used here.
Let us look at a CPU register which is made up of multiple flip flops, each with a capacitance C. Say the register previously had the value `00110101`, and now the value has changed to `10101100`. We can see that 4 bits have flipped, and each transition consumes $E = C \times V^2$, and for 4 bits it will be $E_{total} = 4 \times CV^2$. Since C and V are fixed for a given chip, energy is directly proportional to the number of transitions, which is the hamming distance between the old and new values. Therefore the power consumed is directly proprotaional to the hamming distance and this can be exploited.